

Automated Network Configuration: Architecture and Implementation

Table of Contents

1. Introduction	2
2. JSON Input Format	3
2.1. Interface type discrimination	4
3. Jinja2 Templating	5
3.1. What is Jinja2?	5
3.2. Accessing JSON data in a Jinja2 template	5
3.3. Render invocation	6
4. systemd-networkd Back-end	6
4.1. Overview	7
4.2. File examples produced from the JSON above	7
4.2.1. <code>.link</code> — rename physical interface	7
4.2.2. <code>.netdev</code> — VLAN virtual device	7
4.2.3. <code>.network</code> — parent interface	8
4.2.4. <code>.network</code> — VLAN sub-interface & ROUTE	8
4.3. Execution model	8
4.4. Templates reference	9
5. iproute2 / <code>ip</code> Command Back-end	9
5.1. Overview	9
5.2. Execution model	9
6. Overall Architecture	10
6.1. Component overview	10
6.2. Data-flow walkthrough	11
7. Validation	11
7.1. YANG model validation of the JSON input	11
7.2. Template output integrity checks	11
7.2.1. Structural checks for systemd unit files	12
7.2.2. Script syntax check	12
7.2.3. Content regression tests (Python unittest)	12
7.2.4. Checksum manifest	12
8. Output Structure and OverlayFS Deployment	12
8.1. Why OverlayFS?	13
8.2. Required rootfs-like directory layout	13
9. Jinja2 Practical Reference for Network Templates	13
9.1. Accessing nested YANG data	13

9.2. Handling optional keys safely	14
9.3. String transformation	14
9.4. Cross-interface lookup (parent → VLAN list)	14
9.5. Whitespace control	14
9.6. Standards	15
9.7. Tools and Libraries	15
10. Glossary	15

1. Introduction

This document describes the automated generation of Linux network configuration from structured JSON input files using Jinja2 templates. Two complementary configuration back-ends are covered:

- **systemd-networkd** — the native network daemon of a full **systemd** environment, consuming declarative **.network**, **.netdev**, and **.link** unit files.
- **iproute2 / ip command** — a portable shell-script-based approach suitable for environments where **systemd-networkd** is not available (e.g. a prebuilt rootfs with a stripped-down **systemd**) or another init system is used.

Both back-ends produce the same observable network state: * parent Ethernet interfaces * IEEE 802.1Q VLAN sub-interfaces * static IPv4 addresses * MAC assignments * static IPv4 routes.

The configuration data is modelled in IETF YANG notation (**ietf-interfaces**, **ietf-routing**, ...) and stored as JSON files. Jinja2 templates transform the JSON data into the target configuration artefacts at build time. The output is assembled in a rootfs-like directory tree that can be mounted at run-time via OverlayFS on a read-only base partition.

```
@startuml
left to right direction
rectangle "Source" {
    file "network.json\n(IETF/YANG)" as JSON
}
rectangle "Template Engine" {
    component "Jinja2 render\n(j2yang.py)" as J2
    folder "templates/\n*.j2" as TPL
}
rectangle "Back-ends" {
    folder "systemd-networkd\n*.network *.netdev *.link" as SYSD
    file "ip-config.sh" as IPSH
}
rectangle "Deployment" {
    folder "overlay/\n(rootfs layout)" as OVL
}
JSON --> J2
TPL --> J2
J2 --> SYSD : systemd path
```

```
J2 --> IPSH : iproute2 path
SYSD --> OVL
IPSH --> OVL
@enduml
```



YANG model validation and file-integrity checking are described in [Validation](#).

2. JSON Input Format

All network parameters are stored in a single JSON file per network node. The schema follows the IETF standard data models serialised as JSON:

- `ietf-interfaces:interfaces` ([RFC 8343](#), [YANG source](#)) — parent and VLAN interfaces.
- `ietf-ip` augments for IPv4/IPv6 address configuration ([RFC 8344](#), [YANG source](#)).
- `ietf-if-extensions` for parent-interface and encapsulation fields ([RFC 8795](#), [YANG source](#)).
- `ietf-if-vlan-encapsulation` for dot1q VLAN tagging ([YANG source](#)).
- `ietf-routing:routing` + `ietf-ipv4-unicast-routing:ipv4` ([RFC 8349](#), [YANG source](#), [ipv4-unicast-routing YANG source](#)) — static routes.
- `ietf-system:system` ([RFC 7317](#), [YANG source](#)) — hostname identifying the node.
- `iana-if-type` interface type identities ([IANA ifType registry](#), [YANG source](#)).
- `ietf-if-ethernet-like` for MAC address fields ([RFC 8343 augment](#), [YANG source](#)).

All YANG modules are maintained in the [YangModels/yang GitHub repository](#). Offline copies suitable for `pyang` / `yangson` validation can be downloaded from the [standard/ietf/RFC](#) and [standard/iana](#) directories of that repository.

Example JSON for a network node

```
{
  "ietf-system:system": {
    "hostname": "mpu-node-01"
  },
  "ietf-interfaces:interfaces": {
    "interface": [
      {
        "name": "eth1",
        "type": "iana-if-type:ethernetCsmacd",
        "enabled": true,
        "ietf-if-extensions:max-frame-size": 1500,
        "ietf-if-ethernet-like:ethernet-like": {
          "mac-address": "F2-00-00-00-00-10"
        }
      },
      {
        "name": "eth1.2",
        "type": "iana-if-type:l2vlan",
```

```

    "enabled": true,
    "ietf-if-extensions:max-frame-size": 1500,
    "ietf-if-extensions:parent-interface": "eth1",
    "ietf-if-extensions:encapsulation": {
      "ietf-if-vlan-encapsulation:dot1q-vlan": {
        "outer-tag": {
          "tag-type": "ieee802-dot1q-types:c-vlan",
          "vlan-id": 2
        }
      }
    },
    "ietf-ip:ipv4": {
      "enabled": true,
      "address": [
        { "ip": "10.0.2.16", "prefix-length": 24 }
      ]
    }
  ],
  "ietf-routing:routing": {
    "control-plane-protocols": {
      "control-plane-protocol": [
        {
          "name": "default",
          "type": "ietf-routing:static",
          "static-routes": {
            "ietf-ipv4-unicast-routing:ipv4": {
              "route": [
                {
                  "destination-prefix": "0.0.0.0/0",
                  "next-hop": {
                    "next-hop-address": "10.0.2.1",
                    "outgoing-interface": "eth1.2"
                  }
                }
              ]
            }
          }
        }
      ]
    }
  }
}

```

2.1. Interface type discrimination

The template engine distinguishes interface roles by the **type** field:

type value	Role	Mandatory extra fields
iana-if-type:ethernetCsmacd	Physical / parent Ethernet interface	name, enabled
iana-if-type:l2vlan	IEEE 802.1Q VLAN sub-interface	ietf-if-extensions:parent-interface, iETF-if-extensions:encapsulation

3. Jinja2 Templating

3.1. What is Jinja2?

Jinja2 is a fast, expressive Python template engine. A template is a text file that mixes static content (e.g. INI-file syntax, shell code) with `{{ ... }}` expressions and `{% ... %}` control statements. At render time the engine binds a Python dictionary — the *context* — to the template variables and produces the final text.

Key language constructs used in this project:

Construct	Purpose
<code>{{ variable }}</code>	Emit the value of a variable or expression.
<code>{% for item in list %} ... {% endfor %}</code>	Iterate over a list.
<code>{% if condition %} ... {% elif %} ... {% else %} ... {% endif %}</code>	Conditional rendering.
<code>{{ value filter }}</code>	Apply a filter (e.g. <code>replace</code> , <code>upper</code> , <code>default</code>).
<code>{%- ... -%}</code> (dash trim)	Strip whitespace / newlines adjacent to the tag.
<code>{# comment #}</code>	Template comment (not emitted to output).
<code>{% set name = value %}</code>	Assign a local variable inside the template.

3.2. Accessing JSON data in a Jinja2 template

The Python helper `j2yang.py` loads the JSON file, wraps it under the `data` key if not already present, and passes the resulting dictionary to the `Jinja2 Environment.render()` call.

```
# j2yang.py – context normalisation
def alter_context(context):
    if "data" not in context:
        return {"data": context}
    else:
        return context
```

Inside a template the entire JSON document is therefore accessible as `data`. Nested keys are traversed with the standard Python dict/attribute access:

```

{{# Access the hostname #}}
{{ data['ietf-system:system']['hostname'] }}

{{# Iterate all interfaces #}}
{% for iface in data['ietf-interfaces:interfaces']['interface'] %}
    Name: {{ iface.name }}    Type: {{ iface.type }}
{% endfor %}

{{# Check for an optional key before accessing it #}}
{% if 'ietf-ip:ipv4' in iface and 'address' in iface['ietf-ip:ipv4'] %}
    {% for addr in iface['ietf-ip:ipv4']['address'] %}
        IP: {{ addr['ip'] }}/{{ addr['prefix-length'] }}
    {% endfor %}
{% endif %}

```



Colons inside JSON keys (e.g. `ietf-ip:ipv4`) require bracket notation `data['ietf-ip:ipv4']` rather than dot notation `data.ietf-ip:ipv4`.

3.3. Render invocation

The Python render driver (simplified) looks like:

```

import json
from jinja2 import Environment, FileSystemLoader
from j2yang import alter_context

def render(template_file, json_file, output_file):
    with open(json_file) as fh:
        raw = json.load(fh)

    context = alter_context(raw)

    env = Environment(loader=FileSystemLoader("templates"),
                      keep_trailing_newline=True,
                      trim_blocks=True,
                      lstrip_blocks=True)
    result = env.get_template(template_file).render(**context)

    with open(output_file, "w") as fh:
        fh.write(result)

```

4. systemd-networkd Back-end

4.1. Overview

`systemd-networkd` is the network manager integrated into `systemd`. It reads configuration from three file types placed in `/etc/systemd/network/` (or `/run/systemd/network/` for volatile config):

Extension	Unit type	Purpose
<code>.link</code>	Link configuration	Rename interface, set MAC, MTU at the <code>udev</code> layer (before <code>networkd</code>).
<code>.netdev</code>	Virtual network device	Create virtual devices: VLAN, bridge, bond, etc.
<code>.network</code>	Network configuration	Match an interface and configure IP, VLAN list, routes, DNS, etc.

Files are processed in lexicographic order, so they are prefixed with a two-digit priority number (e.g. `10-eth1.network`, `20-eth1.10.netdev`).

4.2. File examples produced from the JSON above

4.2.1. `.link` — rename physical interface

Generated from `parent_interface_name_static.j2`:

```
[Match]
OriginalName=pfe0
Type=ether

[Link]
Name=eth1
MTUBytes=1500
```

This file causes `udev` to rename the physical NIC (kernel name `pfe0`) to the IETF-configured name `eth1` before `systemd-networkd` starts.

4.2.2. `.netdev` — VLAN virtual device

Generated from `vlan_netdev.j2` for each VLAN interface:

```
[NetDev]
Name=eth1.2

Kind=vlan

[VLAN]
Id=2
```

4.2.3. **.network** — parent interface

Generated from `parent_interface.j2`:

```
[Match]
Name=eth1

[Link]
MACAddress=F2:00:00:00:00:10
MTUBytes=1500

[Network]
VLAN=eth1.2
```

4.2.4. **.network** — VLAN sub-interface & ROUTE

Generated from `vlan_network.j2`:

```
[Match]
Name=eth1.2

[Network]
Address=10.0.2.16/24

[Link]
MTUBytes=1500

[Route]
Gateway=10.0.2.1
Destination=0.0.0.0/0
```

4.3. Execution model

1. The Linux kernel recognises the physical NIC at boot.
2. `udev` applies the `.link` file → interface is renamed to `eth1`.
3. `systemd-networkd.service` reads `/etc/systemd/network/`.
4. For each `.netdev` file, `systemd-networkd` creates the virtual device (e.g. `eth1.2` VLAN sub-interface).
5. For each `.network` file, `systemd-networkd` matches the interface by name and applies the IP / route / MTU / MAC configuration.
6. `systemd-networkd` monitors carrier state and can re-apply configuration on link-up events automatically.

The service dependency order in a typical image:


```

network-pre.target
├── systemd-udevd.service (applies .link files)
│   └── systemd-networkd.service (applies .netdev + .network files)
│       └── network.target

```

4.4. Templates reference

Template file	Output extension	Content
<code>parent_interface_name_static.j2</code>	<code>.link</code>	Renames the physical NIC to the YANG-configured name.
<code>parent_interface.j2</code>	<code>.network</code>	Configures MAC, MTU and VLAN list on the parent interface.
<code>vlan_netdev.j2</code>	<code>.netdev</code>	Creates one VLAN virtual device per VLAN interface.
<code>vlan_network.j2</code>	<code>.network</code>	Assigns IPv4 address, MTU, and static routes to a VLAN interface.
<code>systemd-route.j2</code> / <code>ip-route-zcu.j2</code>	<code>.sh</code>	Shell helper to add routes via <code>ip route replace</code> (platform-specific variant).

5. iproute2 / `ip` Command Back-end

5.1. Overview

The `iproute2` back-end is used on targets where `systemd-networkd` is unavailable — for example, on systems that ship a prebuilt rootfs containing a stripped-down `systemd` without the `networkd` component.

A single shell script (`ip-config.sh`) generated from `ip-config.sh.j2` executes all required `ip` commands in a fixed order:

1. Configure the physical (parent) interface: rename if needed, set MAC, bring up.
2. Create VLAN sub-interfaces.
3. Assign IPv4 addresses.
4. Install static routes.

The script is run by a dedicated `systemd` one-shot service at boot.

5.2. Execution model

1. `systemd` starts `network-config.service` (a one-shot unit with `RemainAfterExit=yes`) early in boot.
2. The service executes `/usr/lib/network-config/ip-config.sh`.

3. Each **ip** command is idempotent — guarded by existence checks — so the script can be re-run safely.
4. The environment variable **VLAN_PARENT** can override the JSON-configured parent name, enabling QEMU testing where interface names differ.

Example systemd unit

```
[Unit]
Description=IP Network Configuration
After=network-pre.target
Before=network.target
Wants=network.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/lib/network-config/ip-config.sh
StandardOutput=journal

[Install]
WantedBy=multi-user.target
```

6. Overall Architecture

6.1. Component overview

```
tools/network-tooling/
├── systemd-network-config/      ← systemd-networkd back-end
│   ├── j2yang.py                ← context normaliser
│   └── templates/
│       ├── parent_interface.j2
│       ├── parent_interface_name_static.j2
│       ├── vlan_netdev.j2
│       ├── vlan_network.j2
│       ├── ip-route-zcu.j2
│       └── systemd-route.j2
│   ├── generate_eth_vlan.sh      ← orchestration shell script
│   ├── test_network.py           ← unit tests for .network templates
│   ├── test_netdev.py            ← unit tests for .netdev templates
│   └── test_vlan_route.py        ← unit tests for route rendering
└── ip-network-config/           ← iproute2 back-end
    ├── j2yang.py
    └── templates/
        └── ip-config.sh.j2
```

6.2. Data-flow walkthrough

```
platform/
├── configuration_end-stations.json  ← multi-node source of truth
│   │
│   │   jq '.[ ] | select(["ietf-system:system"].hostname=="mpu-node-01")'  # see
│   │   https://jqlang.org/manual/
│   │
│   └── fzcu_mpu.json                ← single-node JSON slice
│       │
│       │   generate_eth_vlan.sh  (systemd back-end)
│       │   OR
│       │   j2yang.py + ip-config.sh.j2 (iproute2 back-end)
│       │
│       └── generated artefacts
│           ├── 10-eth1.link
│           ├── 20-eth1.network
│           ├── 20-eth1.2.netdev
│           ├── 30-eth1.2.network
│           └── ip-config.sh
│               │
│               └── assemble into overlay tree
│               │
│               └── overlay/
│                   ├── etc/systemd/network/  ← systemd-networkd files
│                   │   ├── 10-eth1.link
│                   │   ├── 20-eth1.network
│                   │   ├── 20-eth1.2.netdev
│                   │   └── 30-eth1.2.network
│                   ├── usr/lib/network-config/ ← iproute2 script
│                   └── ip-config.sh
```

7. Validation

7.1. YANG model validation of the JSON input

The JSON input files follow the IETF YANG data models. Before rendering, the JSON data should be validated against the YANG schema. Informations and ressources are stored here: https://gittext.elektrobitautomotive.com/dath1233/yang_sdv/tree/master/src/yang_sdv

Example scripts on how to validate the JSON input is located here: https://gittext.elektrobitautomotive.com/dath1233/yang_sdv/tree/master/src/yang_sdv/examples

7.2. Template output integrity checks

After rendering, the generated files should be verified before deployment.

7.2.1. Structural checks for systemd unit files

`systemd-analyze` can inspect unit files for syntax errors:

```
# Verify a .network file
systemd-analyze cat-config systemd/network/20-eth1.network

# Check a .netdev file
systemd-analyze verify --recursive overlay/etc/systemd/network/
```

7.2.2. Script syntax check

```
# POSIX shell syntax check (no execution)
sh -n overlay/usr/lib/network-config/ip-config.sh
```

7.2.3. Content regression tests (Python unittest)

Each test renders a template with a controlled data fixture and asserts the exact expected output:

```
python -m unittest test_network.py    # .network template tests
python -m unittest test_netdev.py     # .netdev template tests
python -m unittest test_vlan_route.py
```

These tests serve as a regression guard: any template change that alters the rendered output will be caught immediately.

7.2.4. Checksum manifest

For deployment integrity, generate a SHA-256 manifest of the overlay tree:

```
find overlay/ -type f -print0 \
| sort -z \
| xargs -0 sha256sum > overlay.sha256

# Verify on target after mount
sha256sum -c /run/network-config-overlay.sha256
```

8. Output Structure and OverlayFS Deployment

8.1. Why OverlayFS?

Embedded Linux images typically use a read-only root partition (A/B or squashfs) to enable atomic updates and protect system integrity. However, network configuration files must reside in `/etc` or `/usr/lib`, which are on the read-only partition.

OverlayFS solves this by stacking a writable *upper* layer over the read-only *lower* layer. The generated network configuration is placed into a dedicated overlay partition that is merged at boot, making the files appear in the expected filesystem locations without modifying the base image.

8.2. Required rootfs-like directory layout

The generated artefacts must be placed in a directory tree that mirrors the absolute paths expected on the target:

```
overlay/                                ← root of the overlay upper layer
├── etc/
│   ├── systemd/
│   │   └── network/                    ← systemd-networkd config files
│   │       ├── 10-eth1.link
│   │       ├── 20-eth1.network
│   │       ├── 20-eth1.2.netdev
│   │       └── 30-eth1.2.network
└── usr/
    └── lib/
        ├── network-config/            ← iproute2 shell script
        └── ip-config.sh
```

9. Jinja2 Practical Reference for Network Templates

Full language reference: [Jinja2 Template Designer Documentation](#).

Installation: `pip install Jinja2` ([PyPI page](#)).

9.1. Accessing nested YANG data

```
{# Get all enabled VLAN interfaces #}
{% for iface in data['ietf-interfaces:interfaces']['interface'] %}
{% if iface.type == "iana-if-type:l2vlan" and iface.enabled %}
    VLAN name : {{ iface.name }}
    Parent    : {{ iface['ietf-if-extensions:parent-interface'] }}
    VLAN ID   : {{ iface['ietf-if-extensions:encapsulation']
                        ['ietf-if-vlan-encapsulation:dot1q-vlan']
                        ['outer-tag']['vlan-id'] }}
{% endif %}
```

```
{% endfor %}
```

9.2. Handling optional keys safely

```
{# Use the 'in' operator before accessing a potentially absent key #}  
{% if 'ietf-if-extensions:max-frame-size' in iface %}  
MTUBytes={{ iface['ietf-if-extensions:max-frame-size'] }}  
{% endif %}  
  
{# Or use the default filter #}  
MTUBytes={{ iface.get('ietf-if-extensions:max-frame-size', 1500) }}
```

9.3. String transformation

```
{# MAC address stored with hyphens in YANG, colon-separated required by ip/systemd #}  
MACAddress={{ iface['ietf-if-ethernet-like:ethernet-like']['mac-address'] | replace('-', ':') }}
```

9.4. Cross-interface lookup (parent → VLAN list)

```
{# Emit VLAN= entries for all VLANs belonging to this parent #}  
{% for vlan in data['ietf-interfaces:interfaces']['interface'] %}  
{% if 'ietf-if-extensions:parent-interface' in vlan  
    and vlan['ietf-if-extensions:parent-interface'] == iface.name %}  
VLAN={{ vlan.name }}  
{% endif %}  
{% endfor %}
```

9.5. Whitespace control

`{%- ... -%}` strips leading/trailing whitespace and newlines. This is critical in INI-format output files where spurious blank lines can confuse parsers:

```
{%- set iface = data['ietf-interfaces:interfaces']['interface'][0] -%}  
{%- if 'ietf-if-extensions:parent-interface' in iface -%}  
[NetDev]  
Name={{ iface.name }}  
{%- endif %}
```

9.6. Standards

Standard	Title
IEEE 802.1Q	<i>Bridges and Bridged Networks</i> — defines 802.1Q VLAN tagging (dot1q).
IANA ifType MIB	Authoritative registry for <code>iana-if-type</code> identity values used in <code>ietf-interfaces</code> (e.g. <code>ethernetCsmacd</code> , <code>l2vlan</code>).

9.7. Tools and Libraries

Tool / Library	Reference
YangModels/yang	Central GitHub repository of all standard IETF, IANA, IEEE, and vendor YANG modules.
Jinja2	Python template engine. <code>pip install Jinja2</code> . Template Designer Documentation .
jq	Lightweight JSON command-line processor used to slice the multi-node configuration JSON.
systemd.network(5)	Man page for <code>.network</code> unit file format.
systemd.netdev(5)	Man page for <code>.netdev</code> unit file format.
systemd.link(5)	Man page for <code>.link</code> unit file format.
ip(8)	iproute2 <code>ip</code> command man page.
Linux OverlayFS documentation	Kernel documentation for the OverlayFS union filesystem.

10. Glossary

Term	Definition
IETF	Internet Engineering Task Force — standards body that defines the YANG data models used.
INI file	A simple key=value configuration file format used by <code>systemd</code> unit files.
iproute2	Linux package providing the <code>ip</code> , <code>ss</code> , <code>tc</code> commands for network management.
Jinja2	Python templating engine; templates are text files with embedded <code>{{ }}</code> and <code>{% %}</code> directives.
NETCONF/YANG	Network Configuration Protocol and its data modelling language used to describe network device state.
OverlayFS	Linux kernel filesystem that transparently merges a read-only lower layer with a writable upper layer.
systemd-networkd	<code>systemd</code> component managing network interfaces via <code>.network</code> , <code>.netdev</code> , <code>.link</code> files.

Term	Definition
VLAN	Virtual LAN — IEEE 802.1Q tagged sub-interface carrying isolated L2 traffic on a shared physical link.
YANG	Yet Another Next Generation — data modelling language (RFC 6020/7950) used to describe NETCONF/RESTCONF data.